

IPL Win Predictor

Tech Stack Document

Every tool explained simply — what it is, why you use it, what you say in interviews

The Golden Rule

Every tool in this project is either something you already know (Python, pandas, sklearn) or something you can learn in one day. Nothing here requires a course. The entire stack runs on your laptop and deploys for free.

1. Core Language

Tool	Version	What It Is	Why You Use It	Interview Answer
Python	3.11	The only programming language in this project	You already know it. All ML tools are Python-first.	Python is the industry standard for ML — every library from sklearn to SHAP is Python-native.

2. Data Tools

Tool	Ver	What It Is	Why Used	What You Say
pandas	2.2	Data manipulation library	Read CSV, create new columns, filter rows, group by team	pandas is how you engineer features — every ML project uses it for data preparation.
numpy	1.26	Numerical computing	Math operations on arrays — used internally by pandas and sklearn	numpy is the foundation under pandas and sklearn — you use it implicitly everywhere.
json (built-in)	stdlib	Parse JSON files	Read Cricsheet JSON match files in <code>parse_cricsheet.py</code>	Cricsheet data comes as JSON — Python's built-in json module parses it with no install needed.

3. Machine Learning Tools

Tool	Ver	What It Is	Why Used	What You Say
scikit-learn	1.4	ML library with 50+ algorithms	Train Logistic Regression, split data, evaluate with <code>classification_report</code>	sklearn is the standard — it provides <code>train_test_split</code> , <code>LogisticRegression</code> , and all evaluation metrics in one package.

XGBoost	2.0	Gradient boosting algorithm	Your final, more accurate model — outperforms Logistic Regression on tabular data	XGBoost handles non-linear feature interactions better than logistic regression — cricket outcomes depend on combinations of runs, wickets, and overs together, not each separately.
imbalanced-learn	0.12	Handles class imbalance	If wins and losses are 60/40 in your data, SMOTE balances training set	IPL data can be slightly imbalanced — SMOTE synthesises minority class samples so the model doesn't just predict the majority class.
SHAP	0.45	Model explainability	Shows which features drove each prediction — visualised as a bar chart in the app	SHAP gives me feature importance per prediction, not just globally. So I can tell a user their win probability dropped because wickets_in_hand fell from 7 to 4 in 2 overs.
joblib	1.3	Save and load the trained model	joblib.dump(model, 'ipl_model.pkl') — saves model to file so app loads it instantly	Joblib serialises numpy arrays efficiently — much faster than pickle for sklearn and XGBoost models.

4. Visualisation Tools

Tool	Ver	What It Is	Why Used	What You Say
matplotlib	3.8	Base plotting library	Draw ROC curves, win % by venue bar charts, probability line charts	matplotlib is the foundation — seaborn and SHAP both use it under the hood.
seaborn	0.13	Statistical visualisation	Prettier charts for EDA — heatmaps, distribution plots, pair plots	Seaborn wraps matplotlib with better defaults — one line of code gives a styled chart.
plotly	5.22	Interactive charts in Streamlit	Charts in the Streamlit app that users can zoom, hover, and explore	Plotly charts are interactive in the browser — users can hover over a point to see the match details, which makes the demo more engaging.

5. App Framework

Tool	Ver	What It Is	Why Used	What You Say
Streamlit	1.35	Python library that builds web UIs	Write Python → get a working web app. No HTML, no CSS, no JS at all.	Streamlit lets me focus on the ML logic rather than frontend code. I write st.slider() and get a UI slider — it abstracts the entire web layer. Given my timeline, this was the right call.

Jupyter Notebook	7.1	Interactive Python environment	Write EDA and model training code with inline charts and markdown explanations	The notebook is part of the deliverable — it documents my thought process, EDA findings, and model decisions. Interviewers can read it like a report.
------------------	-----	--------------------------------	--	---

6. Development & Deployment Tools

Tool	Ver	What It Is	Why Used	What You Say
Git + GitHub	—	Version control + code hosting	Track every change; host public repo that interviewers can read	I used Git with meaningful commit messages per feature — you can see my progress in the commit history.
Streamlit Cloud	Free	Deployment platform for Streamlit apps	Push GitHub repo → get a live public URL in 2 minutes. No server setup.	Streamlit Community Cloud is free for public repos — zero DevOps knowledge needed, which let me spend the deployment week on polishing the app instead.
Antigravity IDE	—	Your code editor	Write and run all Python files locally before deployment	I used [your IDE] for local development — the entire project runs locally with just <code>pip install -r requirements.txt</code> .
requirements.txt	—	Dependency list	List of all pip packages — Streamlit Cloud reads this to install dependencies	One file that makes the project reproducible — anyone can clone and run it in one command.

7. Exact requirements.txt to Copy-Paste

Create a file called requirements.txt in your project root with exactly this content:

```
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.4.2
xgboost==2.0.3
imbalanced-learn==0.12.2
shap==0.45.0
joblib==1.4.2
streamlit==1.35.0
plotly==5.22.0
matplotlib==3.8.4
seaborn==0.13.2
```

Install all at once by running in terminal: `pip install -r requirements.txt`

8. Resume Bullet — Copy This

IPL Win Probability Predictor — Trained XGBoost and Logistic Regression models on 900+ real IPL matches (Cricsheet ball-by-ball data) to predict live win probability. Engineered features including required run rate, wickets in hand, and pressure index. Added SHAP explainability charts. Deployed as Streamlit web app. Stack: Python, pandas, XGBoost, scikit-learn, SHAP, Streamlit. Model accuracy: 80%+.

Skills it adds to your resume: Python · pandas · XGBoost · scikit-learn · SHAP · Streamlit · Feature Engineering · Model Evaluation · Data Visualisation